

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение
высшего образования
«Дальневосточный федеральный университет»

ИНСТИТУТ МАТЕМАТИКИ И КОМПЬЮТЕРНЫХ ТЕХНОЛОГИЙ
(ШКОЛА)

Департамент программной инженерии и искусственного интеллекта

О Т Ч Е Т

о прохождении учебной практики
технологической (проектно-технологической)

Выполнил студент гр. Б9125-09.03.04пи

_____ Рудник А.А.

(подпись)

(ФИО студента)

Отчет защищен с оценкой

Руководитель практики

_____ Ю.Е. Иванова

(должность, уч. степень, уч. звание)

(подпись)

(И.О. Фамилия руководителя)

« 04 » _____ июля 2026 г.

_____ Ю.Е. Иванова

(подпись)

(И.О. Фамилия руководителя)

Регистрационный № _____

« _____ » _____ 2026 г.

(подпись)

(И.О. Фамилия)

Практика пройдена в срок

с « 19 » _____ февраля 2026 г.

по « 04 » _____ июля 2026 г.

на предприятии

ДВФУ

Оглавление

1 Введение	3
2 Основная часть	4
2.1 Описание проекта программного средства	4
2.2 Неформальная постановка задачи	4
2.3 Алгоритм решения задачи	6
2.4 Спецификация данных	9
2.5 Спецификация функций	10
2.6 Алгоритм программы на языке PDL	11
3 Заключение	13
4 Список использованных источников	14

1 Введение

Учебно-технологическая (проектно-технологическая) практика проходила в вузе на базе департамента программной инженерии и искусственного интеллекта Института математики и компьютерных технологий ДВФУ во втором семестре 2025-2026 учебного года с 19.02.2026 г. по 04.07.2026 г. (в объеме 3 з.е., 108 часов).

Целями учебно-технологической (проектно-технологической) практики являются: приобретение первичных практических умений и навыков по разработке проектов программных систем и проектной документации, а также знакомство с профессиональными задачами, решаемыми при создании программных систем.

Задачами учебно-технологической (проектно-технологической) практики являются:

- сбор и анализ требований заказчика к программному продукту;
- формализация предметной области программного проекта по результатам технического задания и экспресс-обследования;
- участие в проектировании компонентов программного продукта в объеме, достаточном для их конструирования в рамках поставленного задания;
- создание компонент программного обеспечения (кодирование, отладка, модульное и интеграционное тестирование);
- разработка тестового окружения, создание тестовых сценариев;
- разработка и оформление эскизной, технической и рабочей проектной документации.

2 Основная часть

2.1 Описание проекта программного средства

Проект программного средства представляет собой 2D survival-игру с элементами обучения математике «Survivors: Math Damage». Игрок перемещается по арене, выбирает числовое значение урона и уничтожает врагов, над которыми отображаются математические примеры. Урон засчитывается только в том случае, если выбранное игроком число совпадает с ответом примера.

Проект разработан с использованием принципов нисходящего проектирования: сначала выделены основные состояния программы, затем описаны данные и функции для каждого состояния, после чего реализованы классы игровых объектов и основной цикл программы. Алгоритм программы основан на объектно-ориентированном и структурном программировании.

Для реализации проекта были использованы язык программирования Python, библиотека Pygame, графические ресурсы в формате PNG/JPG, звуковые ресурсы WAV/MP3 и формат JSON для сохранения прогресса игрока.

Данный краткий отчет оформлен для роли разработчика. В нем рассматривается часть программной реализации, связанная с главным меню, запуском уровней, сохранением прогресса, основным игровым циклом, обработкой ввода, стрельбой и проверкой математического урона. Остальные части проекта, например подробная отрисовка интерфейса, расширенная работа со спрайтами, бонусы, звук, схема проекта или тестирование, могут быть описаны во втором кратком отчете другого разработчика.

2.2 Неформальная постановка задачи

Необходимо разработать образовательную игру, в которой пользователь тренирует устный счет через боевую механику. Игрок управляет персонажем на игровом поле, уклоняется от врагов и стреляет по ним снарядами. Над каждым противником отображается математический пример, а ответ к примеру должен быть выбран игроком как значение урона оружия.

При запуске программы появляется главное меню. В меню пользователь может начать первый уровень, начать второй уровень при наличии доступа, сбросить прогресс или выйти из игры. Первый уровень доступен сразу. Второй

уровень открывается только после успешного прохождения первого уровня, поэтому в меню требуется проверять файл сохранения.

На игровом уровне персонаж управляется клавишами W, A, S, D. Клавиши Q и E изменяют выбранный урон оружия в пределах диапазона текущего уровня. На первом уровне допустимый диапазон урона составляет от 0 до 20, на втором - от 20 до 40. Выстрел выполняется левой кнопкой мыши по направлению к курсору.

Враги появляются за границами экрана и двигаются к персонажу. При попадании снаряда по врагу программа сравнивает урон снаряда с правильным ответом математического примера. Если значения совпадают, здоровье врага уменьшается. Если значения не совпадают, снаряд исчезает, но враг не получает урон.

Игровой процесс ограничен таймером. На первом уровне таймер длится две минуты, на втором - три минуты. После окончания таймера новые враги перестают появляться, а игрок должен уничтожить оставшихся противников. На втором уровне после уничтожения обычных врагов появляется финальный босс.

Во время игры должна быть реализована пауза по клавише Esc. В режиме паузы игрок может продолжить игру, перезапустить уровень, вернуться в главное меню или завершить программу. При поражении отображается экран с результатом и вариантами дальнейших действий. При победе над уровнем прогресс сохраняется.

2.3 Алгоритм решения задачи

1. Начать программу.
2. Инициализировать библиотеку Pygame и звуковую подсистему.
 - 2.1. Получить текущее разрешение экрана пользователя.
 - 2.2. Если разрешение определить невозможно, установить резервный размер окна 1080×720.
 - 2.3. Создать полноэкранное окно без рамки.
 - 2.4. Загрузить изображения интерфейса, кнопок и панелей.
3. Запустить главное меню.
 - 3.1. Загрузить фон главного меню и музыку menu_music.mp3.
 - 3.2. Создать кнопки «Начать 1 уровень», «Начать 2 уровень», «Сбросить прогресс» и «Выход из игры».
 - 3.3. Считать сохранение из save.json.
 - 3.4. Если второй уровень не открыт, сделать кнопку второго уровня неактивной и вывести подсказку об открытии уровня после прохождения первого.
 - 3.5. Если пользователь нажал кнопку первого уровня или клавишу Space, перейти к запуску первого уровня.
 - 3.6. Если пользователь нажал кнопку второго уровня и второй уровень открыт, перейти к запуску второго уровня.
 - 3.7. Если пользователь нажал кнопку сброса прогресса, записать в save.json начальные значения прогресса.
 - 3.8. Если пользователь нажал кнопку выхода или клавишу Esc, завершить программу.
4. Запустить выбранный уровень.
 - 4.1. Очистить группы спрайтов, чтобы уровень начинался без объектов прошлого запуска.
 - 4.2. Создать объект персонажа в центре игровой области.
 - 4.3. Установить номер уровня, начальный счет, таймер уровня и задержку появления врагов.
 - 4.4. Для первого уровня установить длительность 120000 мс и диапазон урона 0-20.
 - 4.5. Для второго уровня установить длительность 180000 мс и диапазон урона 20-40.
5. Выполнять основной игровой цикл уровня.

- 5.1. Считать время кадра dt с ограничением 60 кадров в секунду.
- 5.2. Если игра находится на паузе, обработать кнопки паузы и не обновлять игровые объекты.
- 5.3. Если пользователь нажал W, A, S или D, изменить направление движения персонажа.
- 5.4. Если пользователь отпустил клавишу движения, остановить движение по соответствующей оси.
- 5.5. Если пользователь нажал Q, уменьшить выбранный урон оружия на 1 в пределах диапазона уровня.
- 5.6. Если пользователь нажал E, увеличить выбранный урон оружия на 1 в пределах диапазона уровня.
- 5.7. Если пользователь нажал левую кнопку мыши, создать снаряд в направлении курсора с текущим значением урона.
- 5.8. Если пользователь нажал Esc, открыть меню паузы.
- 6. Обновлять состояние уровня.
 - 6.1. Пока таймер не закончился, уменьшать время до следующего появления врага.
 - 6.2. Если время появления врага истекло, создать нового врага за границей экрана.
 - 6.3. Уменьшать интервал появления врагов по мере прохождения уровня.
 - 6.4. Если таймер уровня закончился, запретить появление новых обычных врагов.
 - 6.5. Обновить движение персонажа, врагов, снарядов и временных эффектов.
- 7. Обработать математическую механику попадания.
 - 7.1. При столкновении снаряда с врагом считать значение $damage$ у снаряда.
 - 7.2. Сравнить $damage$ с $answer$ врага.
 - 7.3. Если $damage = answer$, уменьшить здоровье врага на 1.
 - 7.4. Если $damage \neq answer$, не уменьшать здоровье врага.
 - 7.5. Удалить снаряд после столкновения независимо от результата.
 - 7.6. Если здоровье врага стало меньше либо равно нулю, удалить врага и прибавить к счету значение его ответа.
- 8. Проверять столкновения врага с персонажем.
 - 8.1. Если враг коснулся персонажа и персонаж не находится в режиме неуязвимости, уменьшить здоровье игрока.

- 8.2. Если здоровье игрока меньше либо равно нулю, открыть экран поражения.
- 8.3. Если игрок выбрал рестарт, заново запустить текущий уровень.
- 8.4. Если игрок выбрал меню, вернуться в главное меню.
- 9. Проверять завершение уровня.
 - 9.1. Если таймер закончился и все обычные враги уничтожены, определить номер текущего уровня.
 - 9.2. Если завершен первый уровень, записать в save.json открытие второго уровня и показать экран прохождения уровня.
 - 9.3. Если пользователь нажал «Следующий уровень», запустить второй уровень с сохраненным счетом.
 - 9.4. Если завершен второй уровень и босс еще не создан, создать босса.
 - 9.5. Если босс уничтожен, записать в save.json завершение второго уровня и показать экран прохождения всей игры.
- 10. Если пользователь выбрал выход из игры, закрыть окно и завершить программу.

2.4 Спецификация данных

В рамках роли разработчика используются следующие входные данные.

1. Пользовательский ввод: клавиши W, A, S, D для движения персонажа; Q и E для изменения выбранного урона; Space для запуска первого уровня; 2 для запуска второго уровня; Esc для паузы или выхода; левая кнопка мыши для нажатия кнопок и выполнения выстрела.

2. Данные экрана: width и height - ширина и высота окна; GAME_AREA_HEIGHT - высота игровой области без нижней информационной панели; INFO_PANEL_HEIGHT - высота нижней панели.

3. Данные уровня: current_level - номер уровня; LEVEL_DURATIONS - длительность уровней; SPAWN_DELAYS - начальная и конечная задержка появления врагов; STRONG_ENEMY_CHANCE - вероятность появления сильного врага.

4. Данные игрока: health - текущее здоровье; max_health - максимальное здоровье; dx и dy - направление движения; speed - скорость; weapon_damage - выбранный урон; min_damage и max_damage - границы допустимого урона; last_shot_time - время последнего выстрела.

5. Данные врага: health - здоровье врага; max_health - максимальное здоровье; example - математический пример; answer - правильный ответ; is_strong - признак сильного врага; is_boss - признак босса.

6. Данные снаряда: damage - урон, выбранный игроком; dx и dy - направление движения снаряда; speed - скорость движения; god_mode - признак тестового режима.

7. Данные сохранения: save.json хранит логические значения level_2_unlocked и level_2_completed. Если файл отсутствует или поврежден, используются значения по умолчанию.

Выходные данные, формируемые данной частью программы: открытие главного меню, запуск выбранного уровня, изменение счетчика очков, изменение здоровья игрока и врагов, сохранение прогресса, переход на экран прохождения уровня, переход на экран поражения или завершение программы.

2.5 Спецификация функций

1. terminate() - завершает работу Pygame и закрывает программу.

2. `load_progress()` - загружает данные сохранения из `save.json` и возвращает состояние открытия второго уровня.
3. `save_progress(level_2_unlocked, level_2_completed)` - обновляет файл сохранения после прохождения уровней.
4. `reset_progress()` - сбрасывает прогресс и закрывает доступ ко второму уровню.
5. `reset_groups()` - очищает группы спрайтов перед запуском или перезапуском уровня.
6. `start_screen()` - отображает главное меню, обрабатывает выбор уровня, сброс прогресса и выход из программы.
7. `run_game(level, score_at_level_start)` - запускает игровой цикл выбранного уровня и возвращает результат работы уровня.
8. `main()` - управляет общими переходами между главным меню, уровнями, рестартом, следующим уровнем и выходом.
9. `Character.move(direction, movement)` - изменяет направление движения персонажа при нажатии или отпускании клавиши.
10. `Character.change_damage(delta)` - изменяет выбранный урон оружия, не выходя за границы диапазона уровня.
11. `Character.shoot(target_pos, current_time)` - создает снаряд по направлению к курсору мыши с учетом задержки между выстрелами.
12. `Attack.update()` - перемещает снаряд, проверяет столкновение с врагом и применяет математическую проверку `damage = answer`.
13. `spawn_enemy(level)` - создает обычного или сильного врага за границей экрана.
14. `get_spawn_delay(level, level_elapsed, level_duration)` - рассчитывает текущую задержку появления врагов в зависимости от прогресса уровня.
15. `end_screen(count, level)` - отображает экран поражения и возвращает выбранное пользователем действие.
16. `level_complete_screen(level, count)` - отображает экран успешного прохождения первого уровня.
17. `game_complete_screen(count)` - отображает экран полного прохождения игры после победы на втором уровне.

2.6 Алгоритм программы на языке PDL

Ниже приведен фрагмент алгоритма, относящийся к индивидуальной части разработчика: обработка меню, запуск уровня, пользовательский ввод и проверка математического урона. Комментарии описывают назначение действий.

```
procedure MAIN()
  // главный цикл приложения
  while game_is_running do
    menu_action = START_SCREEN()
    if menu_action == "quit" then
      TERMINATE_PROGRAM()
    endif
    if menu_action == "level2" then
      current_level = 2
    else
      current_level = 1
    endif
    total_score = 0
    repeat
      result, total_score = RUN_GAME(current_level, total_score)
      if result == "restart" then
        continue current level
      endif
      if result == "next_level" then
        current_level = 2
        continue next level
      endif
      if result == "menu" then
        exit to main menu
      endif
    until result == "menu" or result == "quit"
  endwhile
endprocedure

procedure RUN_GAME(level, score_at_level_start)
  // подготовка уровня
  RESET_GROUPS()
  character = CREATE_CHARACTER(level)
  count = score_at_level_start
  level_elapsed = 0
  timer_finished = false
  boss_spawned = false
  while level_is_active do
    dt = CLOCK_TICK(60)
    READ_EVENTS()
    if key == ESC then
      OPEN_PAUSE_MENU()
    endif
    if key in {W, A, S, D} then
      CHARACTER_MOVE(key)
    endif
    if key == Q then
      CHANGE_DAMAGE(-1)
    endif
  endwhile
endprocedure
```

```

        if key == E then
            CHANGE_DAMAGE(+1)
        endif
        if mouse_left_click then
            SHOOT_TO_MOUSE(character.weapon_damage)
        endif
        UPDATE_TIMER_AND_SPAWN_ENEMIES()
        UPDATE_SPRITES()
        CHECK_ATTACK_COLLISIONS()
        CHECK_PLAYER_HEALTH()
        CHECK_LEVEL_FINISH()
        DRAW_GAME_SCENE()
    endwhile
endprocedure

procedure CHECK_ATTACK_COLLISIONS()
    // математическая проверка попадания
    for each attack in attack_sprites do
        enemy = COLLIDE(attack, enemy_sprites)
        if enemy exists then
            if attack.damage == enemy.answer then
                enemy.health = enemy.health - 1
                if enemy is boss and enemy.health > 0 then
                    REFRESH_BOSS_EXAMPLE()
                endif
            endif
            DELETE(attack)
        endif
    endfor
endprocedure

```

3 Заключение

В процессе прохождения практики выполнены следующие задачи:

- проведен сбор и анализ требований заказчика к программному средству;
- выполнена формализация выбранной предметной области;
- разработана и реализована часть программного средства, связанная с главным циклом игры, обработкой пользовательского ввода, переходами между экранами и математической механикой урона;
- реализована работа с сохранением прогресса;
- проведена отладка основных пользовательских сценариев;
- разработана и оформлена отчетная документация по индивидуальной роли разработчика.

Таким образом, цели учебно-технологической (проектно-технологической) практики достигнуты.

В результате прохождения практики получены следующие профессиональные компетенции:

- способен разрабатывать языковые процессоры для формальных языков (ПК-7);
- способен создавать программные интерфейсы (ПК-8);
- способен использовать различные технологии разработки программного обеспечения (ПК-10).

4 Список использованных источников

1. McGugan W. Beginning Game Development with Python and Pygame: From Novice to Professional. - Apress, 2007. - 336 с.
2. Sweigart A. Making Games with Python & Pygame. - 2012. - 366 с.
3. Официальная документация Python [Электронный ресурс]. - Режим доступа: <https://docs.python.org/3/> (дата обращения: 11.06.2026).
4. Официальная документация Pygame [Электронный ресурс]. - Режим доступа: <https://www.pygame.org/docs/> (дата обращения: 11.06.2026).
5. Python Software Foundation. Модуль json [Электронный ресурс]. - Режим доступа: <https://docs.python.org/3/library/json.html> (дата обращения: 11.06.2026).
6. ГОСТ 19.701-90. Единая система программной документации. Схемы алгоритмов, программ, данных и систем. - М.: Стандартиформ, 1990. - 26 с.
7. ГОСТ 19.402-78. Единая система программной документации. Описание программы. - М.: Стандартиформ, 1978. - 8 с.

ДНЕВНИК СТУДЕНТА

Дата	Рабочее место	Краткое содержание выполняемых работ	Отметки руководителя
19.02.2026	ДВФУ	Вводное занятие по технике безопасности, обсуждение этапов выполнения практики	выполнено
05.04.2026-12.04.2026	ДВФУ	Изучение требований к образовательной игре и распределение задач между разработчиками	выполнено
19.04.2026-26.04.2026	ДВФУ	Настройка проекта Python/Pugame, создание структуры файлов и каталогов ресурсов	выполнено
26.04.2026-03.05.2026	ДВФУ	Реализация главного меню и обработки кнопок запуска уровней	выполнено
03.05.2026-10.05.2026	ДВФУ	Реализация системы сохранения прогресса в файле save.json	выполнено
10.05.2026-17.05.2026	ДВФУ	Реализация основного игрового цикла первого уровня	выполнено
17.05.2026-24.05.2026	ДВФУ	Реализация обработки движения персонажа и выбора урона оружия	выполнено
24.05.2026-31.05.2026	ДВФУ	Реализация стрельбы и проверки математического ответа через значение урона	выполнено
01.06.2026-08.06.2026	ДВФУ	Интеграция второго уровня и условий перехода между уровнями	выполнено
08.06.2026-15.06.2026	ДВФУ	Отладка меню паузы, рестарта и возврата в главное меню	выполнено
15.06.2026-.06.2026	ДВФУ	Подготовка PDL-фрагмента и краткого отчета по роли разработчика	выполнено
29.06.2026 — 04.07.2026	ДВФУ	Защита отчета по практике	выполнено

Студент

(подпись)

(ФИО студента)

Руководитель практики

(подпись)

(ФИО руководителя)